Aarhus university

Computer-Science

Numerical Linear Algebra

Handin 5

Author
Søren M. Damsgaard

Student number
202309814

March 17, 2026

The 'Nice' moment of using SVD

(a) - Plotting a Curve

Part (a) and (b) is trivial plotting and rotation which has been done multiple times now, but the code can be found in the appendix where it is neatly divided into each question.

$$x = 4 \cdot \cos(t) \qquad y = \sin(2 \cdot t) \quad \text{for } 0 \leq t \leq 2 \cdot \pi \tag{1}$$

The author shall name this figure 'OptimusPrime'

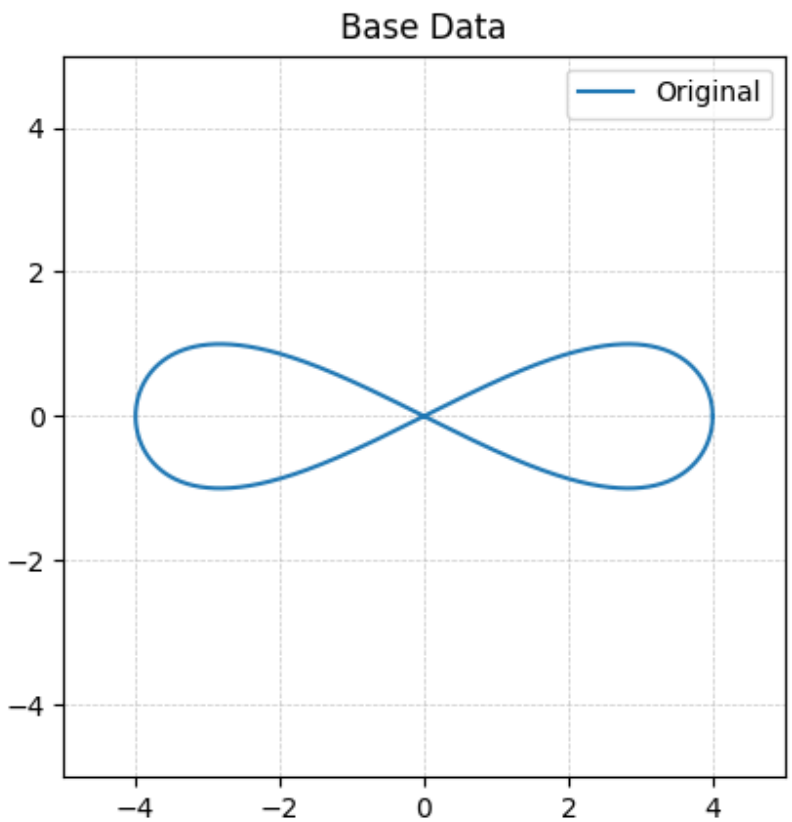

Figure 1: Plotting the function $x = 4 \cdot \cos(t)$ and $y = \sin(2 \cdot t)$

(b) - Rotating the Curve

The author shall now use the power of magical math to rotate OptimusPrime.

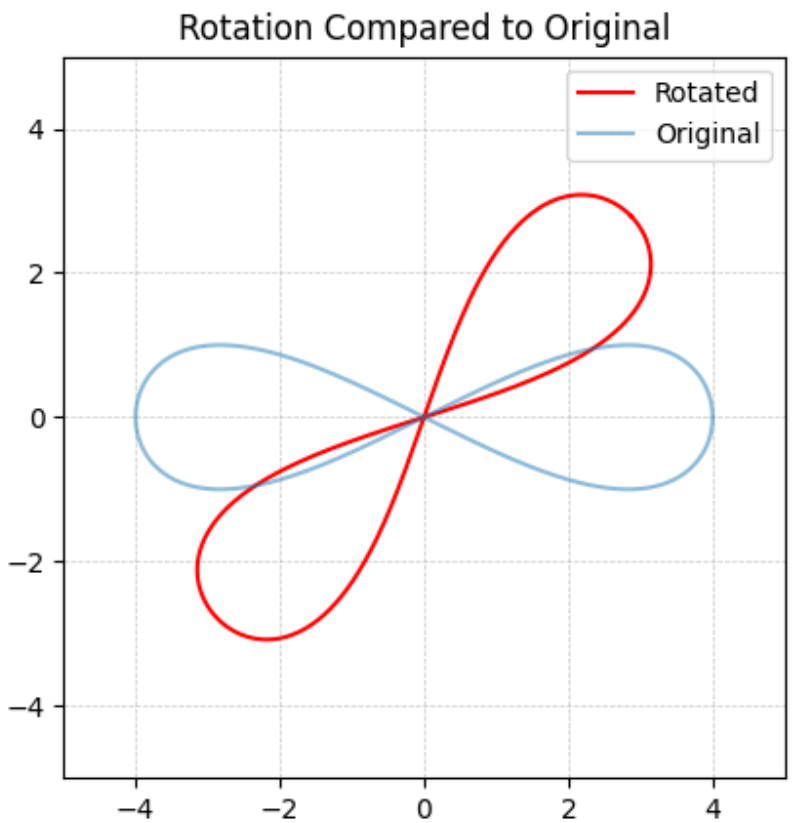

Figure 2: The figure OptimusPrime and its rotation Megatron

(c) - Adding Noise

This is done the same way as in Assignment 2, but the noise is made out to be dots for a better visual and comparison.

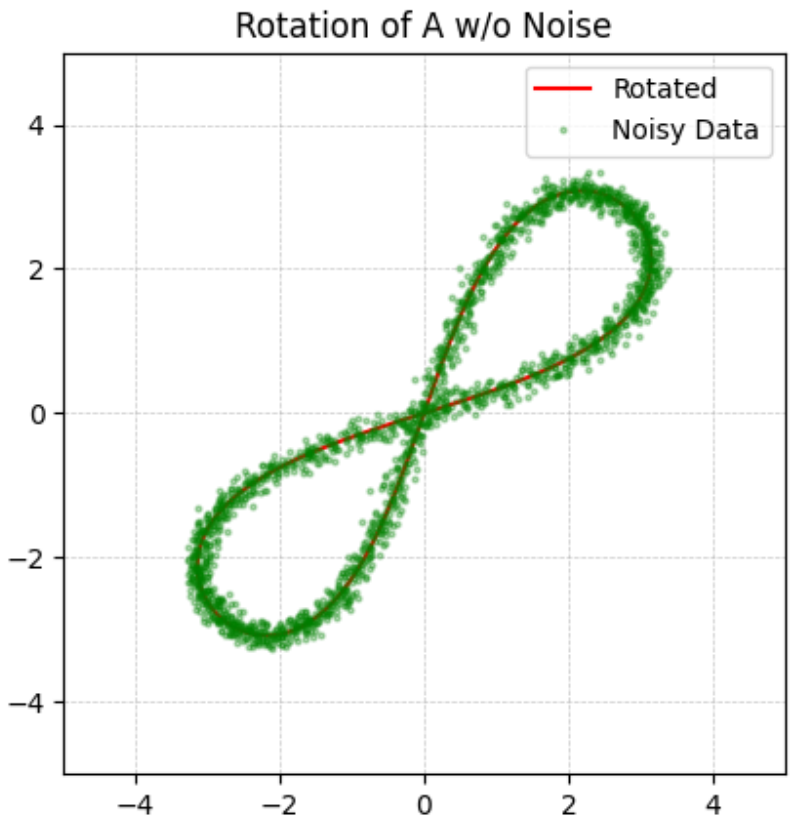

Figure 3: Adding noise to all entries in, comparing it to before and after

(d) - New Matrix B(aby)

From here on out, it's new territorie and the author can no longer call on The Law of Triviality.
Refere to Line 27-30 for the code to this part.
By subtracting the mean of A with itself, the resulting matrix B is only a displacement of A, centering it at the origin, which is alleged to be important when factorizing B with SVD later.
Although for this case, it almost seems redundant to do so, since the figure already 'looks' to be centered at the origin, but it's important to account for the noise potentially ofcentering the figure, hence this step is still essential.
For comparison in this case the mean of both A and B

$$\text{mean of A} = [-0.0026, 0.0025] \qquad \text{mean of B} = [0.\ 0.]$$

(e) - Computing SVD of B

Refere to LINE 31-35
As is described in the book, the SVD can be computed with U, S, VT = NP.LINALG.SVD(), which in this case gives:

$$u = \begin{pmatrix} -0.69 & 0.73 \\ 0.73 & 0.69 \end{pmatrix} \qquad s = \begin{pmatrix} 126.56 & 0 \\ 0 & 32.01 \end{pmatrix}$$

To verify that u is orthogonal it has to hold that $u^T u = I$ which can be shown:

$$\begin{pmatrix} -0.69 & 0.73 \\ 0.73 & 0.69 \end{pmatrix} \begin{pmatrix} -0.69 & 0.73 \\ 0.73 & 0.69 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

So u is orthogonal

(f) - Describing Geometry with SVD

SVD is essentialy the factorization of a matrix where each variable of the SVD describes a part of the matrix, more specifically u(left singular vectors) and s(singular values) describes the geometry while vt is the right singular vectors.
Even more specifically u is a 2×2 matrix on the form $[u_0|u_1]$ where the column vector u_0 describes the direction on where the variance is the largest and u_1 describes the direction where the variance is the smallest, note since u is orthogonal the lenght of both u_0 and u_1 is 1.
This is where s comes in! Whereas u describes the direction of thte variance, the singular points in s describes the magnitude of the variance.
So the singular values describe how much the figure 'stretches' while the left singular vectors is the direction of the stretch.

(g) - Restoring B to the Original

See LINE 42
This is where it gets interesting, since OptimusPrime(Original) is symmetrical (just look at it), the maximum variance described by u and s can also be assumed to be symmetrical which means if you rotate u it's the same as rotating the whole figure.
To make B look like OptimusPrime we have to align it with the axis, and we just argued that u is essentially responsibly for the rotation, which means that if we inversely rotate B with u^T it should aling wiht the x and y axis.

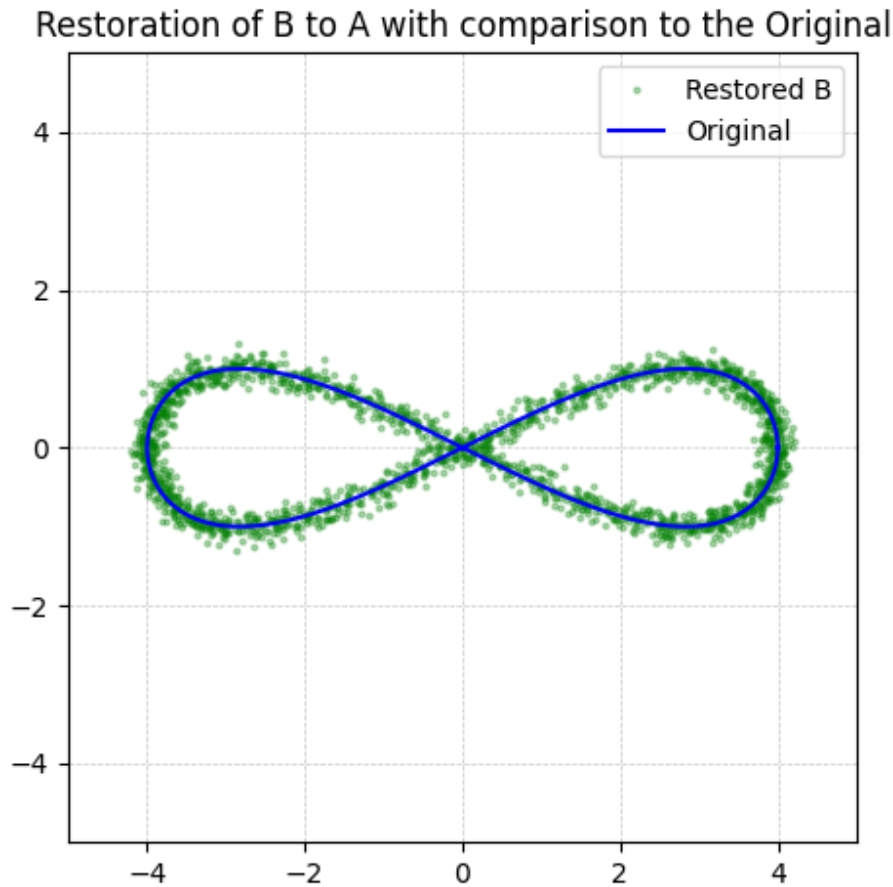

Figure 4: Rotation of matrix B approximating it to the shape of the Original figure from (a)

Note: The author cooked HARD with this last part, but the author is known to be a bad cook, so please apply first-aid(feedback) if needed.

Appendix

```
import numpy as np
import matplotlib.pyplot as plt


# Defining variables
c = np.cos
s = np.sin
pie = np.pi
rng = np.random.default_rng()
theta = rng.uniform(pie/7, 6*pie/7)
# Rotation, getRotated
R = np.array([[c(theta),-s(theta)],
                      [s(theta),c(theta)]])
n = 2000 # Reasonable large n
```

```python
noise = rng.normal(0.0, 0.1, (2, n))

# (a)
t = np.linspace(0, 2*pie, n)
x = 4*c(t)
y = s(2*t)
# (b)
points = np.array([x, y])
pointsR = R @ points
# (c)
A = pointsR + noise

# (d)
meanA = np.mean(A, axis=1, keepdims=True) # If not true it will output sadness
B = A - meanA
print('The mean of matrix B: \n',np.mean(B, axis=1)) # Close enough to be considered zero
# (e)
u, s, vt = np.linalg.svd(B)
print('The left singular vector: \n',u)
print('The singular values of B from SVD:\n',np.diag(s))
print('If the following is the identity matrix (true), U is Orthogonal: \n', np.allclose(u.T @ u, np.eye(2))) #BRO IT'S ZEROO!! RARWAAW!
# it's floating point precision type shiz, so it is the identity matrix, trust.

# (f)
# This is a theoretical part which will be coverede in the PDF, not the code

# (g)
restoredB = u.T @ B




#---Plotting evil functions(plans)---
fig1, curve = plt.subplots()
curve.plot(x, y, label='Original')
curve.set_title('Base Data')


fig2, curveRot = plt.subplots()
curveRot.plot(*pointsR, label='Rotated', color='red')
curveRot.plot(x, y, label= 'Original', alpha=0.5)
curveRot.set_title('Rotation Compared to Original')

fig3, curveRotNoise = plt.subplots()
curveRotNoise.plot(*pointsR, label='Rotated', color='red')
curveRotNoise.plot(*A, 'o', markersize=2, label='Noisy Data', color='green', alpha=0.3)
curveRotNoise.set_title('Rotation of A w/o Noise')

```

```python
fig4, jesus = plt.subplots()
jesus.plot(*restoredB,'o', markersize=2, label='Restored B', color='green', alpha=0.3)
jesus.plot(x,y, label='Original', color='blue')
jesus.set_title('Restoration of B to A with comparison to the Original')

# Asked AI to find make my plots equal, in regards to their X-Y axis
for ax in [curve, curveRot, curveRotNoise, jesus]:
    ax.grid(True, which='both', linestyle='--', linewidth=0.5, alpha=0.7)
    ax.set_xlim(-5, 5)
    ax.set_ylim(-5, 5)
    ax.set_aspect('equal')
    ax.legend()

plt.tight_layout()
plt.show()
```